Analysis of iOS SQLite Schema Evolution for Updating Forensic Data Extraction Tools

Samiha S. Shimmi
Department of Computer Science & Engineering
Ahsanullah University of Science & Technology
Dhaka, Bangladesh
samiha_14.cse@aust.edu

Gokila Dorai
School of Computer & Cyber Sciences
Augusta University
Augusta, Georgia
gdorai@augusta.edu

Umit Karabiyik
Department of Computer and Information Technology
Purdue University
West Lafayette, Indiana
umit@purdue.edu

Sudhir Aggarwal
Department of Computer Science
Florida State University
Tallahassee, Florida
sudhir@cs.fsu.edu

***Abstract*—Files in the backup of iOS devices can be a potential source of evidentiary data. Particularly, the iOS backup (obtained through a logical acquisition technique) is widely used by many forensic tools to sift through the data. A significant challenge faced by several forensic tool developers is the changes in the data organization of the iOS backup. This is due to the fact that the iOS operating system is frequently updated by Apple Inc. Many iOS application developers release periodical updates to iOS mobile applications. Both these reasons can cause significant changes in the way user data gets stored in the iOS backup files. Moreover, approximately once every couple years, there could be a major iOS release which can cause the reorganization of files and folders in the iOS backup. Directories in the iOS backup contain SQLite databases, plist files, XML files, text files, and media files. Android/iOS devices generally use SQLite databases since it is a lightweight database. Our focus in this paper is to analyze the SQLite schema evolution specific to iOS and assist forensic tool developers in keeping their tools compatible with the latest iOS version. Our recommendations for updating the forensic data extraction tools is based on the observation of schema changes found in successive iOS versions.**

***Index Terms*—SQLite schema, iOS app forensics, iOS data extraction, iOS SQLite database evolution, Mobile forensics**

I. Introduction

There are two kinds of applications (apps) that run in an iPhone: native apps and third-party apps. The database schema of iPhone apps can change when the iOS version is updated or/and when individual apps are updated. App updates can run either automatically or be initiated by a user.

Forensic data extraction from iOS devices based on logical extraction is heavily dependent on how data is organized in the file system in the form of SQLite database files, plist / XML files, media, and text files. SQLite databases are most commonly used by various apps in the iOS backup to store data. Often times, it becomes a challenge to maintain mobile forensic data extraction tools since this file organization keeps changing due to mobile operating system updates or app version updates. The forensic tool developers have limited knowledge about the changes made to the SQLite schema of the iOS backup, and they have to keep their tools compatible with the app version changes. Hence there is a necessity to identify such changes and modify the data extraction tools (especially the SQL queries in the source code), which is a tedious and time-consuming task. To address this challenge, we have developed a tool named "SQLite Database Comparison Analyzer" (SDCA) which is capable of analyzing the differences between two versions of SQLite schemas automatically. SDCA also has query analysis capability. In addition to developing the SDCA tool, we have practically demonstrated the use of the tool in order to update a Forensic Targeted Data Extraction Application [1] called FTDEA developed by the authors.

The rest of the paper is organized as follows. In Section II, we have discussed the related work. In Section III, we have discussed the background of iOS backups and the importance of analyzing the database schemas used in iOS backups in order to update third-party forensic data extraction tools. We have performed the analysis using three progressive phases, termed pre-analysis, static analysis, and dynamic analysis. Each of these phases is explained in Section IV along with the high-level system design of SDCA. Finally, we conclude the paper with discussion of some future work in Section V.

II. Related Work

Various researchers have published their work on locating various artifacts in the iOS backup structure for forensic analysis. Bader et al. [2] explored the logical forensic acquisition of the iPhone using a free backup utility provided by Apple. The authors discussed and analyzed the backup data, and also documented the forensically relevant backup files and locations. This research can help a user understand the basic structure of the backup folder.

978-1-7281-6939-2/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

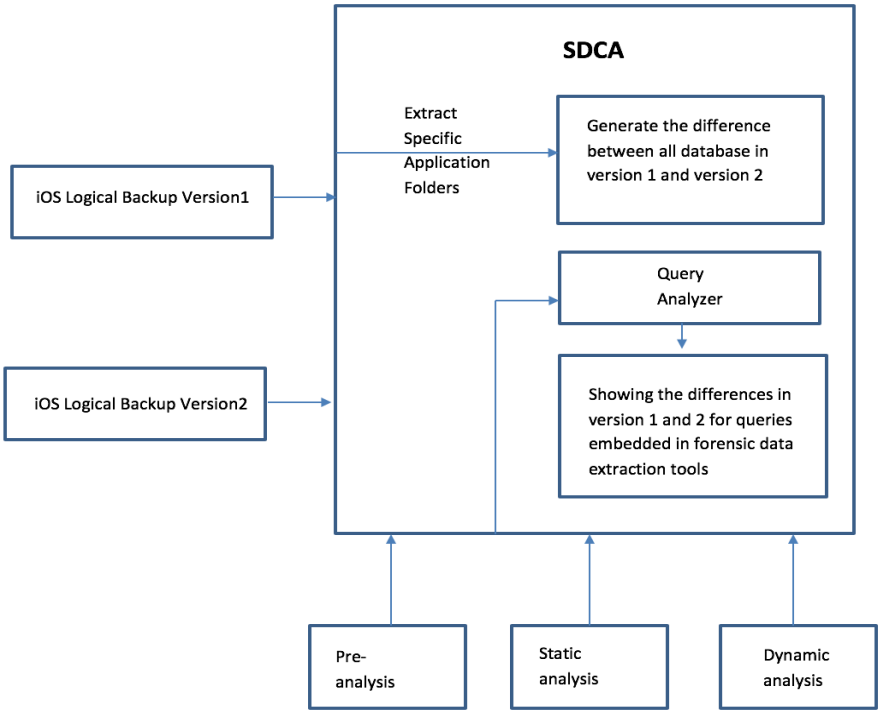

Fig. 1. High-level System Design of SDCA

Carpene [3] and Piccinelli et al. [4] have explored and analyzed the iOS backup files obtained using iTunes and their work lists the available tools used to analyze iOS backup files. While this work helps us understand the data stored in the iOS backup, there has been less work done on analyzing the schema evolution of databases used in the iOS backup as per our knowledge.

Cleve et al. [5] analyzed a large project called OSCAR(Open Source Clinical Application Resource). They designed a system to get SQL queries from the software repository and used a SQL parser to obtain the logical schema from SQL queries. With that information, they developed a global historical schema from all previous versions of the database. OSCAR has sensitive information, and analyzing *data* to get further information about the table (like table renames, table merges, etc.) was difficult to perform due to restrictions. The authors were particularly interested in one specific project and wanted to help people to understand the schema evolution of their database as many third-party developers use that database.

Prism [6] is a handy commercial tool applied to the Wikipedia project with the MYSQL database. Bounif et al. [7] discusses a predictive approach for database evolution. However, in the case of an iOS backup, we do not know what kind of information gets stored in which field and which table just by analyzing the schema decomposition.

Lin et al. [8] used empirical evidence to identify challenges and solutions associated with the collateral evolution of application programs and databases. By collateral evolution, they mean potential inconsistencies that arise when application and database evolution is not synchronized. Lin et al. performed both schema and file format evolution studies on two open-source programs. They identified the challenges that occurred by the collateral evolution of applications and databases. Compared to our work, their work has focused on a different problem.

The researchers in other domains, rather than iOS, did some schema evolution analysis. Wu et al. [9] performed a long-term of schema evolution research on four open-source applications. They wrote schema extractors for C, C++, and Objective-C. They extracted SQLite schema from code and converted them into MySQL format. They used a comparison tool, which worked for MySQL. Wu et al. showed the schema evolution history of those four applications.

Meurice et al. [10] presented DAHLIA, a visual analyzer of database schema evolution. They extracted SQL code from the Git/SVN repository of four projects. They extracted the schema and retrieved historical information about the schema. Compared to these studies, we aim to help the developers by providing a tool which could help them compare SQLite databases, more specifically, iOS backup databases. We have designed our tool in such a way that providing two iOS backup versions will yield comparison-results at the schema level.

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

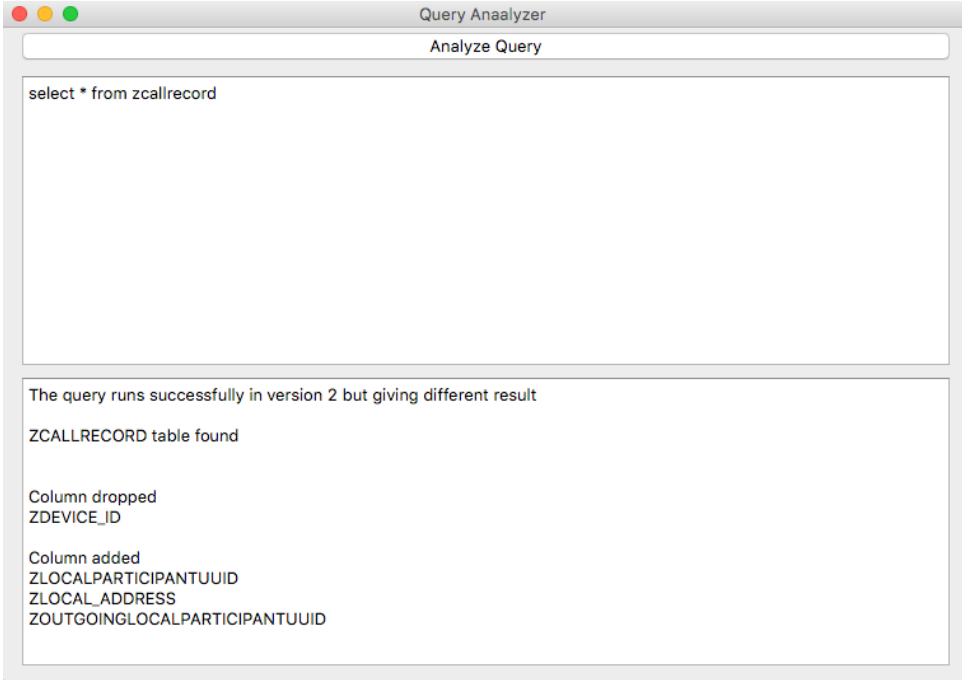

Fig. 2. Screenshot of the Output from Query Analyzer

Our tool will also help any third-party developers who utilize the iOS backup files. Also, we studied the schema evolution history of iOS backups, which help developers to determine the changes caused due to iOS backup schema changes.

Qiu et al. [11] conducted an empirical analysis of the co-evolution of schema and code. They analyzed ten open-source projects and answered some research questions on schema evolution. They extracted details about DB revisions from the code repository. Curino et al. [12] discuss their work on the Wikipedia schema evolution.

Recently, Bonifati et al. [13] worked on schema evolution for graph databases whereas Möller et al. [14] worked on noSQL database. Arthinaiou et al. [15] introduced VESEL, a system enabling the visual exploration of schema evolution which can efficiently answer queries about when, how and why the schema changes occur. They worked on top of BiDeL [16]. To our knowledge, SDCA is the first automation tool to work on the schema evolution of iOS backup and this tool can be very useful for the forensic community for keeping their data extraction tools up-to-date.

Users generally do not have any information about database schemas of apps present in the iOS backup. Therefore, in this study we focused on understanding such schema differences caused due to iOS version updates and iOS app version updates.

III. Background

In this section, we discuss the background of iOS backups. iTunes is an application from Apple Inc., which is used to synchronize the content of iPhone, iPad, or other Apple devices. iTunes is also used to obtain an encrypted / unencrypted backup of iOS device(s) on computers. Such a backup resides in a fixed location for Windows OS and Mac OS computers. For example, in a Mac OS computer, the iOS backup is located in the following location: `/Library/Application Support/MobileSync/Backup/`. For the analysis, we obtain the iOS backup using iTunes utility on MacBook. The name of the backup folder is a string of 40 hexadecimal digits and it represents a unique identifier for the device. Inside this backup directory, there are sub-directories. Each of these sub-directories may contain one or more SQLite databases. The files inside the backup directory can be categorized as follows:

- SQLite database files
- Plain text plist files
- Binary plist files
- Multimedia and text files
- Non-standard data files

In this paper, we have analyzed the evolution of SQLite database files in particular due to their popularity and extensive use across several embedded platforms. SQLite databases use a lightweight framework, and they are popular in the embedded community. SQLite database is viewed as a complete SQL database with multiple tables, views, and indices that can be stored in a single disk file [17].

When we analyzed the iOS backup, we found that the name of each SQLite database file is represented by 40-digit hexadecimal characters in a similar way like iOS backup folder names. The name of each file is the hashed value (using the SHA1 algorithm) of the location of the file in the file system [3]. We have analyzed four different iOS versions (iOS v9, iOS v10, iOS v11, and iOS v12) and found that the names of SQLite databases of native applications were the same for all

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

four versions.

IV. High Level System Design

Fig. 1 shows the high-level design of the SDCA tool. It takes two iOS backup versions. We use an internal SQLite database to store information about those backups. SDCA is a GUI tool that provides a user interface to help the user select the databases they want to compare and analyze.

Our system consists of three sequential phases: *pre-analysis*, *static analysis* and *dynamic analysis*. The SDCA tool is a deliverable of the pre-analysis phase. In the static and dynamic phases, we integrate and test the SDCA tool on forensic tools that use static and dynamic database queries for data extraction. Python and PyQt5 were used to build SDCA.

A. Pre-Analysis

In the pre-analysis part, we analyzed four consecutive versions of the iOS backup. In this phase, the SDCA tool takes the given iOS backup as input and lists the various SQLite database files from the backup directories. SDCA is programmed to do the following steps sequentially:

1) SDCA extracts SQLite database names and file path information from the iOS backups and stores this information in a local database.
2) When users select a specific SQLite database from the list, SDCA will generate a text file which contains all the changes identified in the schema structure from one iOS version to the other.
3) Further, depending on the type of forensic tool and the list of SQLite queries used by the forensic tool for data extraction from databases in the iOS backup, SDCA invokes a Query Analyzer that then parses the input query and lists the differences (see an example in Fig. 2).

SDCA can detect seven different types of schema changes in SQLite databases based on queries such as 'add table', 'delete table', 'rename table', 'add column', 'drop column', 'rename column' and 'data-type' changes. When we give an iOS backup as input to SDCA, it stores all the information in an internal database that contains tables to store the SQLite information, FTDEA (the forensic tool developed by the authors) query information, and table information. The schema information of each table in two iOS backup versions is fetched and compared using this internal database. In special cases such as a table or a column that are renamed, it becomes a challenge to identify such a change in different versions of SQLite tables in the iOS backup. In order to address this problem, we compare the same data in two different iOS versions using SDCA. Backups taken before and after updating the iPhone operating system are used for this type of analysis. Such an analysis helps us understand how the same information from apps could get stored in two different iOS versions. SDCA will check whether any new table was added to the database as a replacement for another table. If so, the data is compared between the two tables. If a match is found, then the table is marked as 'renamed.' The same process is followed for column renaming as well. The table and column renaming algorithms used for developing SDCA are shown as Algorithms 1 and 2.

Algorithm 1 Find table renaming

```
for each dropped table dropT do
    result1 ← execQuery("select * from dropT")
    for each added table addT do
        result2 ← execQuery("select * from addT")
        if result1 == result2 then
            table renamed
        end if
    end for
end for
```

Algorithm 2 Find column renaming

```
Define T as table name
for each dropped column dropC do
    result1 ← execQuery("select dropC from T")
    for each added column addC do
        result2 ← execQuery("select addC from T")
        if result1 == result2 then
            column renamed
        end if
    end for
end for
```

As mentioned before, four different iOS versions, such as iOS 9, 10, 11, and 12, were analyzed. Each time, we analyzed two consecutive versions using SDCA. For some applications, we also analyzed the data present in the SQLite files. This analysis helped us understand what has changed in the database due to the iOS version updates. For instance, if a column was dropped in the new iOS version, we verified why it was dropped and how the operating system has accommodated the data in the tables of the new iOS version. With this type of analysis, we were able to make some design decisions on how to program SDCA to help third-party forensic tool developers to write a new data extraction query and make their tool compatible with the latest iOS version.

For analysis purposes, we chose some of the most commonly used iOS applications such as iMessage, Contact, Call Log and Calendar apps. These are also the apps from which most forensic tools extract valuable data about a user/suspect/victim. The list of files we analyzed is listed in Table I. Table II summarizes the analysis on all four database.

In order to calculate the change rate (CR) and growth rate (GR) of database schemas, we have used the following two formulas ((1) and (2)). Reference to these formulas can be found in [11]. Table III shows the growth rate and change rate of the schemas. From this table, we can observe that the GR and CR for SMS schema was 56% when the iOS version was updated from 10 to 11. The GR and CR values are high for Call Log schema when the iOS version was updated from 11 to 12.

$$\text{GR} = \frac{\text{Added Elements(AE) - Deleted Elements(DE)}}{\text{Number of Initial Elements(IE)}} \tag{1}$$

$$\text{CR} = \frac{\text{Added Elements(AE) + Deleted Elements(DE)}}{\text{Number of Initial Elements(IE)}} \tag{2}$$

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LIST OF ANALYZED FILES

Database	Database name (hash value) in the iOS backup
SMS	3d0d7e5fb2ce288813306e4d4636395e047a3d28
Address book	31bb7ba8914766d4ba40d6dfb6113c8b614be442
Call log	5a4935c78a5255723f707230a451d79c540d2741
Calendar	2041457d5fe04d39d0ab481178355df6781e6858

TABLE II
NUMBER OF TABLES IN EACH VERSION OF IOS

Database	No of tables in iOS 9	No of tables in iOS 10	No of tables in iOS 11	No of tables in iOS 12
SMS	9	9	14	14
Address book	28	29	29	29
Call log	5	5	5	7
Calendar	34	36	36	37

TABLE III
GROWTH AND CHANGE RATE OF SCHEMA SIZE

		Tables					Columns				
Database	**iOS Version**	**IE**	**AE**	**DE**	**GR**	**CR**	**IE**	**AE**	**DE**	**GR**	**CR**
SMS	9-10	9	0	0	0%	0%	93	13	0	14%	14%
	10-11	9	5	0	56%	56%	107	24	0	22%	22%
	11-12	14	0	0	0%	0%	147	3	0	2%	2%
Contact	9-10	28	3	2	4%	18%	163	28	17	7%	28%
	10-11	29	0	0	0%	0%	187	1	0	1%	1%
	11-12	29	0	0	0%	0%	188	1	0	1%	1%
Call Log	9-10	5	0	0	0%	0%	34	3	0	9%	9%
	10-11	5	0	0	0%	0%	37	3	0	0%	0%
	11-12	5	2	0	40%	40%	37	1	1	0%	0%
Calendar	9-10	34	2	0	6%	6%	370	10	0	3%	3%
	10-11	36	0	0	0%	0%	390	14	2	3%	4%
	11-12	36	1	0	3%	3%	402	0	0	0%	0%

B. Static Analysis

We have programmed a static analyzer for parsing the FTDEA forensic tool written in Java. This static analyzer is part of the SDCA tool and can parse any forensic tool which has static queries with no dynamic variables used in the query. We have to feed the SDCA tool with the location of the source code of the forensic tool and the queries retrieved will be shown on the GUI of the SDCA tool. Fig. 3 shows a screenshot of various queries extracted by the static analyzer of SDCA.

In order to detect the SQL queries, the database access information must be identified in the source code of forensic tools. The FTDEA tool is capable of analyzing SMS, Call Log, and Contact apps. In FTDEA, the JDBC framework is used to access the database. The parser takes the FTDEA source code as input. The parser then obtains the information about all SQL queries used in the source code along with the line number and file names. In the case of a forensic tool that uses JDBC, the string inside the "executequery()" function is extracted. In FTDEA, the queries were hardcoded without any string concatenation. So, just extracting the query string inside the function, as mentioned above, was sufficient for further analysis.

C. Dynamic Analysis

Almost all forensic tools used for extracting data from iOS devices use simple to complex SQLite queries. We discuss how query extraction might not be simple in some cases. For instance, authors in [18] explored some widely used database access Java frameworks. They mentioned JDBC, Spring, JPA, Vaadin-GWR and Hibernate as the most popular frameworks. These frameworks provide a language-specific abstraction layer which is a strong feature. On the other hand, because of this abstraction, it becomes harder to retrieve the queries manually.

For a large software project in a real-life scenario, we might deal with several dynamic queries. An approach, such as the static analysis will not be helpful in such a scenario. As an example, Meurice et al. [19] performed static analysis on three different open-source Java applications, such as Oscar, OpenMRS, and Broadleaf. JDBC, Hibernate, and JPA were used by these applications in order to access the database. Meurice et al. pointed out the challenges of retrieving all the SQL queries. For all these three open-source systems

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

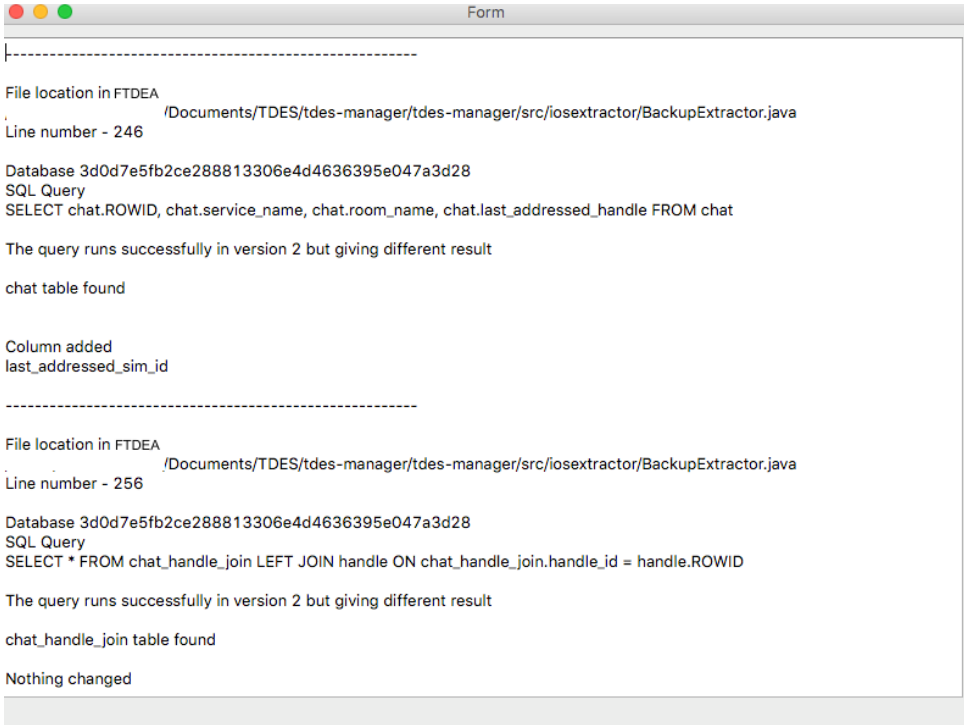

Fig. 3. The Output of FTDEA Application Analysis Using SDCA

mentioned, the rate of successfully retrieved queries is 71.5, 82.4, and 99%, respectively. Also, the percentage of valid queries for each system is 94.9, 87.9, and 100%, respectively. The static nature of the analysis tool is a limitation when we deal with dynamic queries. We cannot retrieve all the queries without running the applications and even if we get queries they are not always valid queries as shown by the above work.

The above reasons were the impetus for testing SDCA's ability to handle run-time queries. Instead of providing the source code of a forensic tool to the static analyzer of SDCA, we decided to integrate the SDCA tool as part of the forensic tool itself. As a result, queries can be fed to the SDCA tool at run-time during the execution of a forensic data extraction tool. In order to test this, the SDCA tool was slightly changed by removing the Graphical User Interface and simply converting it into a library file. Since it is written in Python, any Python-based application would be able to use the SDCA easily. We ran the tests by integrating SDCA as part of FTDEA. The file name, line number, database name and SQL query were fed to SDCA dynamically.

V. Conclusion and Future Work

Digital forensic tools play a vital role in forensic investigations, especially during the examination and analysis phases. In these phases forensically relevant information is retrieved by using tools mostly in an automated fashion. This automation is simply performed based on the knowledge gathered by tool developers or researchers after identifying the data structures and locations in file systems. Through the operating system updates this knowledge can become easily obsolete hence the tools become malfunctioning. As a result, forensic tools may not be validated by investigators and it also poses serious burden on the investigators.

In this paper we discussed and introduced SQLite Database Comparison Analyzer (SDCA). We strongly believe that this work will help forensic tool developers to deal particularly with the problem of iOS updates and iOS schema evolution. The easy-to-use and cross-platform nature of the SDCA tool will help various third-party forensic tool developers in understanding iOS version updates and for analyzing the database queries used in the source code of forensic data extraction tools. As future work, we aim to integrate the Query Rewriting tool as part of SDCA and also test the adaptability of SDCA for a tool like Autopsy or other open source tools.

VI. Acknowledgement

This project was supported in part by an extension to Award No. 2016-MU-CX-K003, awarded by the National Institute of Justice, Office of Justice Program, U.S. Department of Justice. The opinions, findings and conclusions or recommendations expressed in this publication/program/exhibition are those of the author(s) and do not necessarily reflect those of the Department of Justice.

References

[1] S. Aggarwal, G. Dorai, U. Karabiyik, T. Mukherjee, N. Guerra, M. Hernandez, J. Parsons, K. Rathi, H. Chi, T. Aderibigbe, *et al.*, "A targeted data extraction system for mobile devices," in *IFIP International Conference on Digital Forensics*, pp. 73–100, Springer, 2019.

[2] M. Bader and I. Baggili, "Iphone 3gs forensics: Logical analysis using apple itunes backup utility," *Small Scale Digital Device Forensics Journal*, vol. 4, pp. 1–15, 01 2010.

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.

[3] C. Carpene, "Looking to iphone backup files for evidence extraction," 01 2011.
[4] M. Piccinelli and P. Gubian, "Exploring the iphone backup made by itunes," *JDFSL*, vol. 6, no. 3, pp. 31–62, 2011.
[5] A. Cleve, M. Gobert, L. Meurice, J. Maes, and J. Weber, "Understanding database schema evolution," *Sci. Comput. Program.*, vol. 97, pp. 113–121, Jan. 2015.
[6] C. A. Curino, H. J. Moon, and C. Zaniolo, "Graceful database schema evolution: The prism workbench," *Proc. VLDB Endow.*, vol. 1, pp. 761–772, Aug. 2008.
[7] H. Bounif and R. Pottinger, "Schema repository for database schema evolution," pp. 647–651, 01 2006.
[8] D.-Y. Lin and I. Neamtiu, "Collateral evolution of applications and databases," *International Workshop on Principles of Software Evolution (IWPSE)*, 01 2009.
[9] S. Wu and I. Neamtiu, "Schema evolution analysis for embedded databases," in *2011 IEEE 27th International Conference on Data Engineering Workshops*, pp. 151–156, April 2011.
[10] L. Meurice and A. Cleve, "Dahlia: A visual analyzer of database schema evolution," in *2014 Software Evolution Week - IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE)*, pp. 464–468, Feb 2014.
[11] D. Qiu, B. Li, and Z. Su, "An empirical analysis of the co-evolution of schema and code in database applications," in *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2013, (New York, NY, USA), pp. 125–135, ACM, 2013.
[12] C. A. Curino, L. Tanca, H. J. Moon, and C. Zaniolo, "Schema evolution in wikipedia: toward a web information system benchmark," in *In ICEIS*, 2008.
[13] A. Bonifati, P. Furniss, A. Green, R. Harmer, E. Oshurko, and H. Voigt, "Schema validation and evolution for graph databases," in *International Conference on Conceptual Modeling*, pp. 448–456, Springer, 2019.
[14] M. L. Möller, M. Klettke, and U. Störl, "Keeping nosql databases up to date - semantics of evolution operations and their impact on data quality," in *LWDA*, 2019.
[15] C. Athinaiou and H. Kondylakis, "Vesel: Visual exploration of schema evolution using provenance queries.," in *EDBT/ICDT Workshops*, 2019.
[16] K. Herrmann, H. Voigt, A. Behrend, J. Rausch, and W. Lehner, "Logical data independence in the 21st century–co-existing schema version support with inverda,"
[17] S. Consortium, "What is sqlite?." https://www.sqlite.org/, 2020.
[18] M. Goeminne and T. Mens, "Towards a survival analysis of database framework usage in java projects," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 551–555, Sep. 2015.
[19] L. Meurice, C. Nagy, and A. Cleve, "Static analysis of dynamic database usage in java systems," in *Advanced Information Systems Engineering* (S. Nurcan, P. Soffer, M. Bajec, and J. Eder, eds.), (Cham), pp. 491–506, Springer International Publishing, 2016.

Authorized licensed use limited to: National Central University. Downloaded on December 23,2020 at 09:49:37 UTC from IEEE Xplore. Restrictions apply.